



The LDBC Social Network Benchmark

Gábor Szárnyas, Ben Steer, Jack Waudby, Peter Boncz

with contributions from former members of the SNB Task Force

The LDBC Social Network Benchmark

- Initial mission: developing the LDBC Benchmark suite
 - Design started in 2012, in the making for ~10 years
 - Influential in both academia and industry
- From 2015: new LDBC groups around query languages
 - Query Language Task Force (G-CORE)
 - GQL Formal Semantics Working Group
 - Existing Languages Working Group
 - Property Graph Schema Working Group
- LDBC has a growing membership and an active online community

This presentation is a summary of the LDBC Social Network Benchmark. We provide an overview of the benchmark and codify some lessons learnt during the last 5 years.

For an overview of the LDBC, see the talk titled [“The Linked Data Benchmark Council”](#)

Data sets

- Graph schema
- Correlated data
- Deletions
- The Datagen project

Social network domain

Disclaimer: We acknowledge that running queries on a social network is *not the primary use case* of graph databases.

However: It is a widely understood domain with interesting graph data structures. Additionally, it makes it easy to argue regarding correlations in the data such as:

- “People in *Germany* are more likely to be called *Joachim* than in *Italy*”
- “People in the *France* make more trips to *Belgium* than people in *Mexico* to *Japan*”

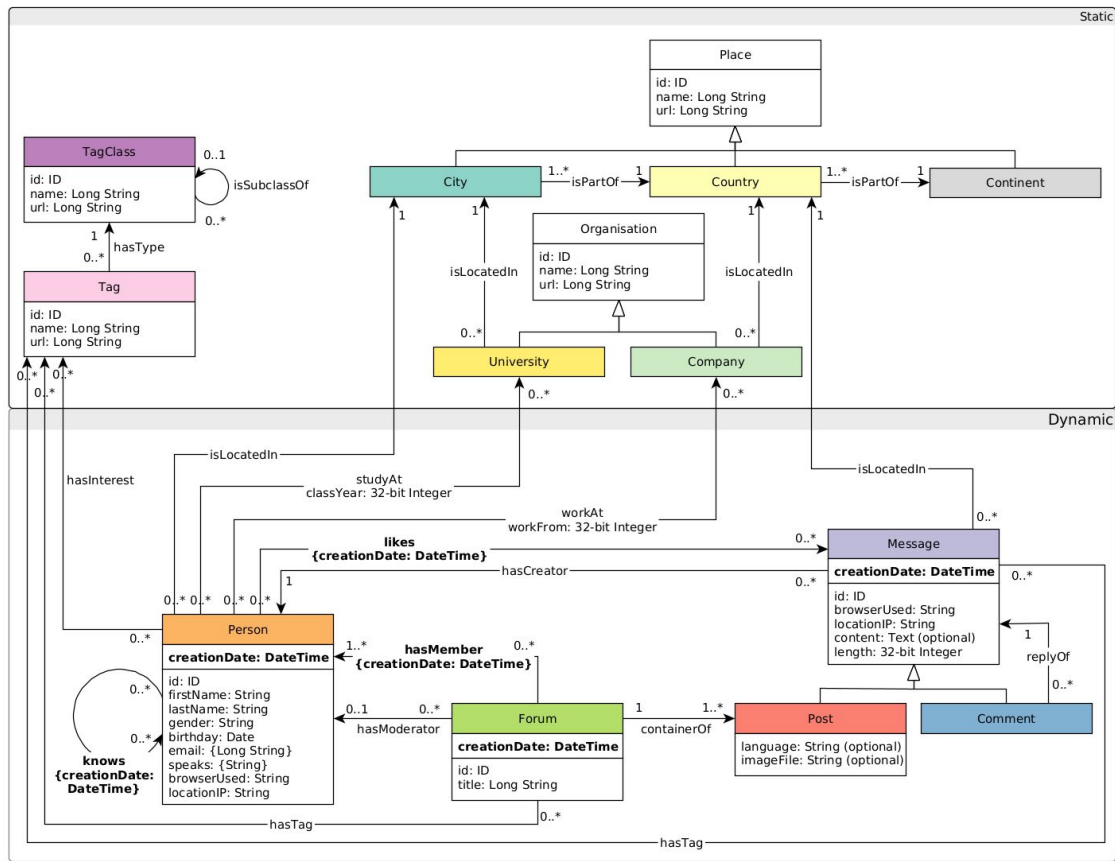
The generated graphs are realistic *to some extent* but not fully. The goal is to add some realistic correlations which query engines can exploit.

Statistics

Network of Person nodes, trees
of Messages/TagClasses/Places

Statistics for scale factor 1:

- 3M nodes, 17M edges
- 11k Persons, avg. degree of knows edges: 39.4
- Branching factors
 - Message tree: 3.2
 - TagClass tree: 3.7
 - Place tree: 12.4



Graph schema

The graph is a **labelled property graph**. All edges are directed except the Person-knows-Person edges, which are *undirected*.

(To avoid duplications, the undirected edges are only serialized to one direction and it is the responsibility of the loader/query implementation to treat them as undirected.)

Edge types (between node types) can be categorized as follows:

- Bipartite: most edge types form a bipartite subgraph, e.g. Forum-hasMember-Person
- Network: Person nodes form network along the knows edges
- Hierarchies:
 - TagClasses: a rooted tree of TagClass nodes (root: “Thing”)
 - Places: a non-rooted tree of 3 levels (Continent, Country, City)
 - Messages: each thread is a rooted tree with a Post root node and Comment nodes

Data generator (Datagen)

Produces a **property graph** data set

The graph is fully dynamic: **inserts** and **deletes** with realistic distributions

Distributed generation for scalability:

- Based on Hadoop (2012-2020), used for the Interactive workload
- Migrated to Spark in 2020, used for the BI workload

 [S3G2: a Scalable Structure-correlated Social Graph Generator](#), TPCTC 2012

 [LDBC SNB Datagen: Under the hood by Arnau Prat](#), 9th LDBC TUC meeting, 2017

 [Supporting dynamic graphs in SNB Datagen](#) by J. Waudby et al., GRADES-NDA 2020

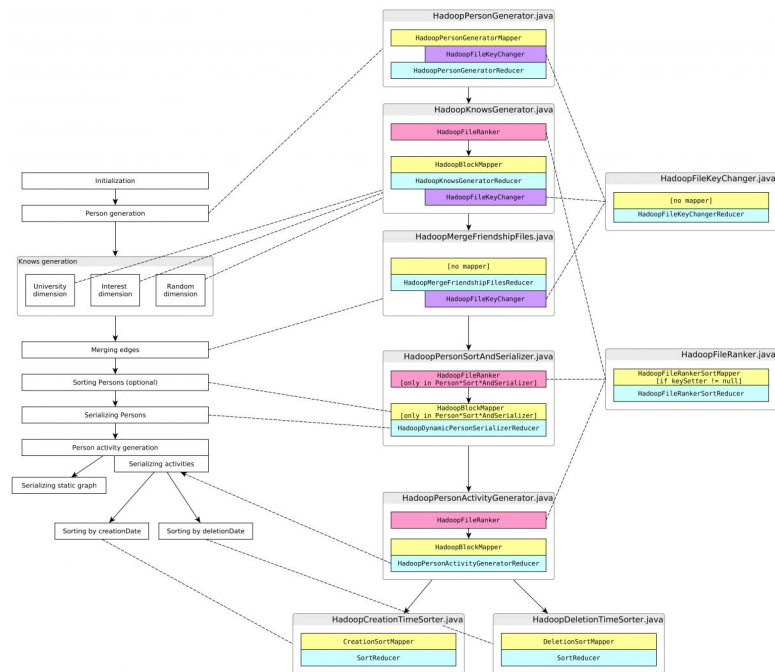
 [Speeding up LDBC SNB Datagen](#), blogpost, 2020

Data generator (Datagen)

Graphs are produced using a distributed data processing framework

- Earlier versions used Hadoop
- Migrated to Spark in 2020

Capable of producing output with different serializers (CSV variants, Turtle).



Refresh operations

The “dynamic” part of the graph is changing throughout the benchmark. This puts systems using static data structures (such as plain CSR) at a disadvantage.

Depending on the workload of SNB, the refresh operations are different:

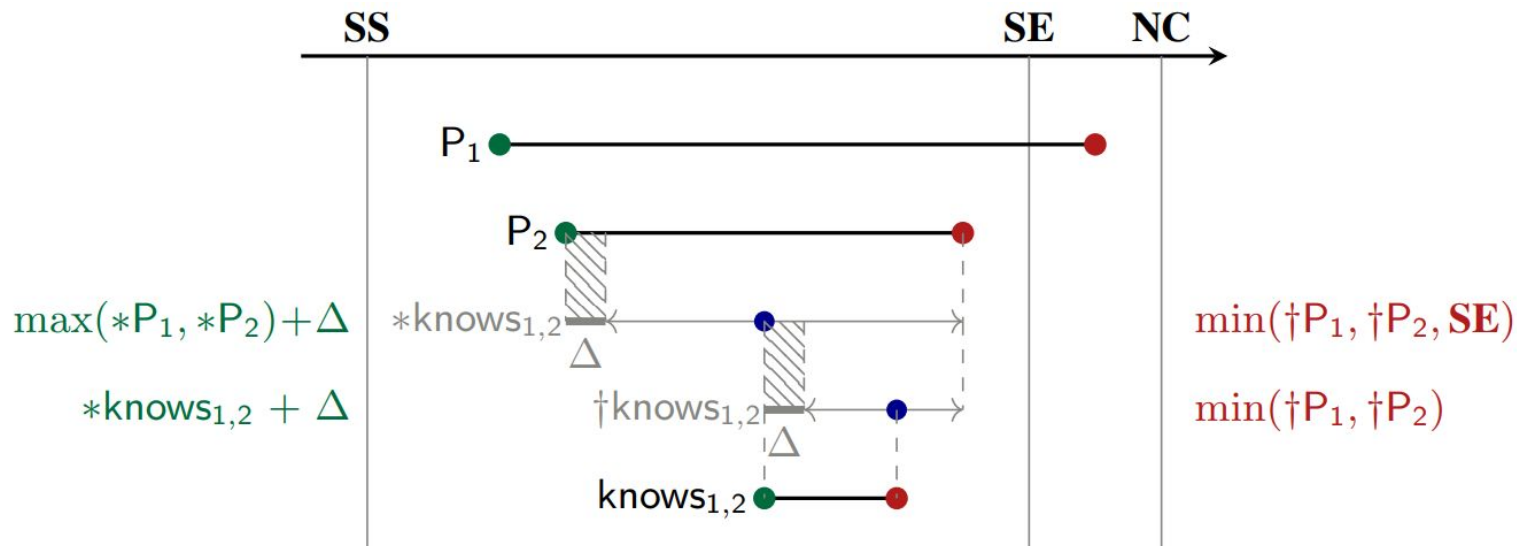
Interactive: New Persons/Forums/Messages are *inserted* along with their edges

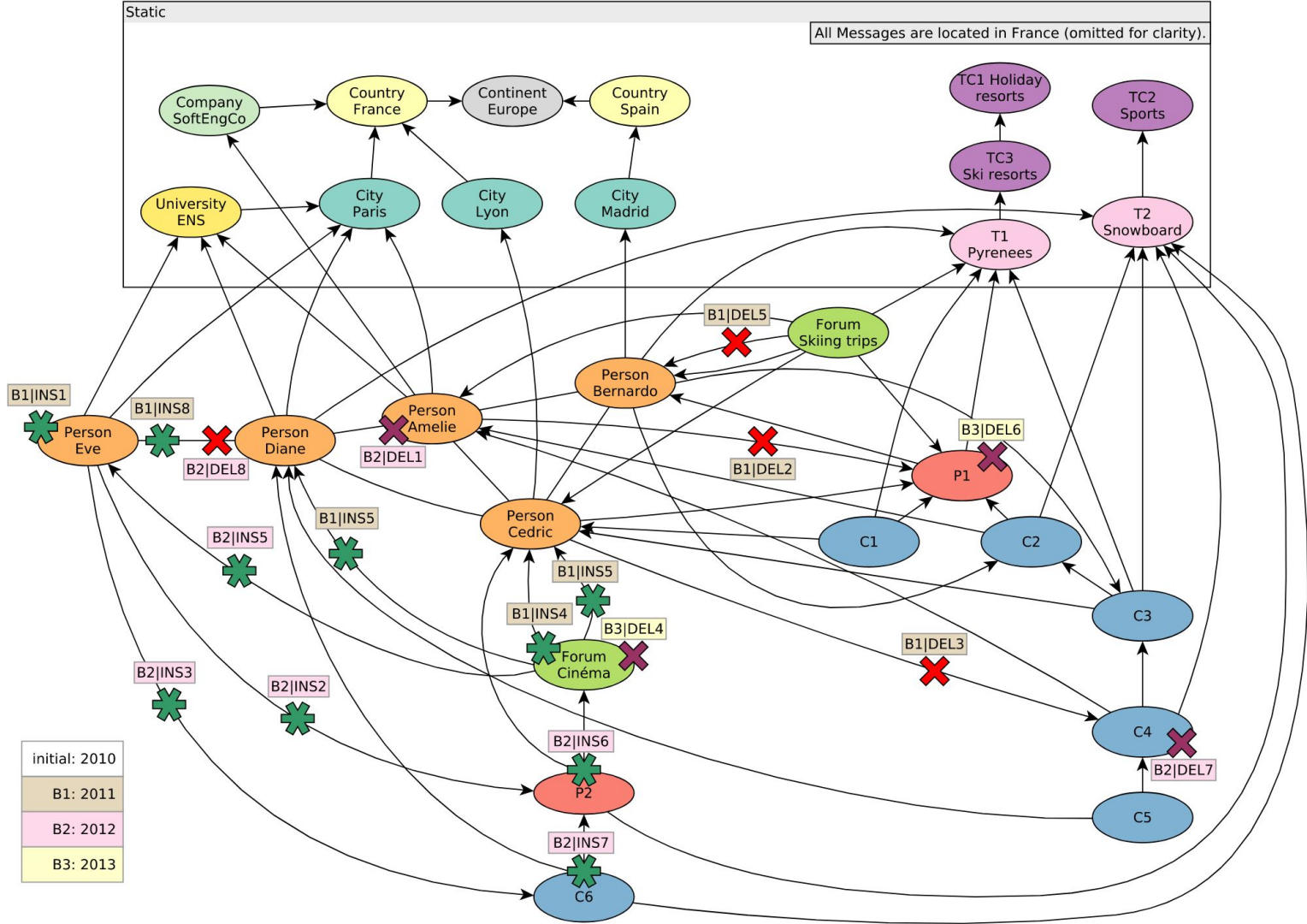
BI: Same *inserts* plus the same type of entities are also subject to *deletes*

Generating deletions is challenging as it necessitates lifespan management, cascading deletes (e.g. deleting an entire Forum or a Message thread), and has a significant impact on the distribution of the data.

Lifespan management

When can a Person-knows-Person edge exist? Its **creation date* and *†deletion date* values are selected from intervals constrained by those of the Person endpoints.





Workloads

- Interactive workload
- BI workload

Comparison of workloads

	Interactive	BI
focus	OLTP	OLAP
results	always non-empty	can be empty
typical query	2-3 hop neighbourhood queries with filtering	multi-hop/path/subgraph queries with filtering & aggregation
refresh operations	inserts (<i>called “updates”</i>)	inserts and deletes
ACID compliance	<i>required</i>	
data set	<i>social network graph</i>	

Interactive workload

—

Interactive workload

Scenario: Users browsing a social network and producing content (Forums, Messages)

Queries: 14 complex reads, 7 short reads, 8 insert operations

Audit rules:

- Imperative implementations are allowed
- Defining materialized views is allowed (the tradeoff is maintenance cost)

 Recently completed: first audited benchmark, TuGraph by FMA Technologies [[FDR](#)]

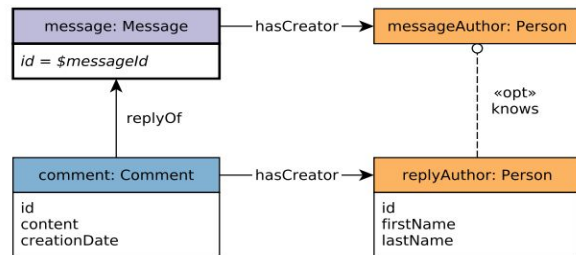
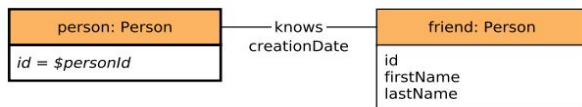
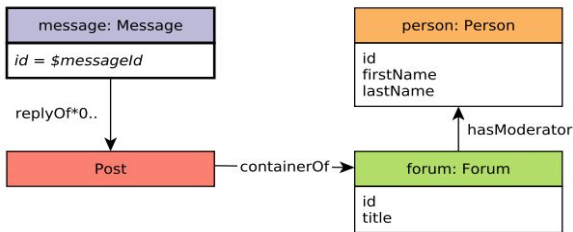
 SNB Interactive [[SIGMOD'15 paper](#)] [[SIGMOD'15 slides](#)]

 [Benchmark page](#)

Interactive workload: Queries

Complex queries: Always start from one or two Person nodes, and discover their neighbourhoods (1..2 nodes) or paths between Person nodes.

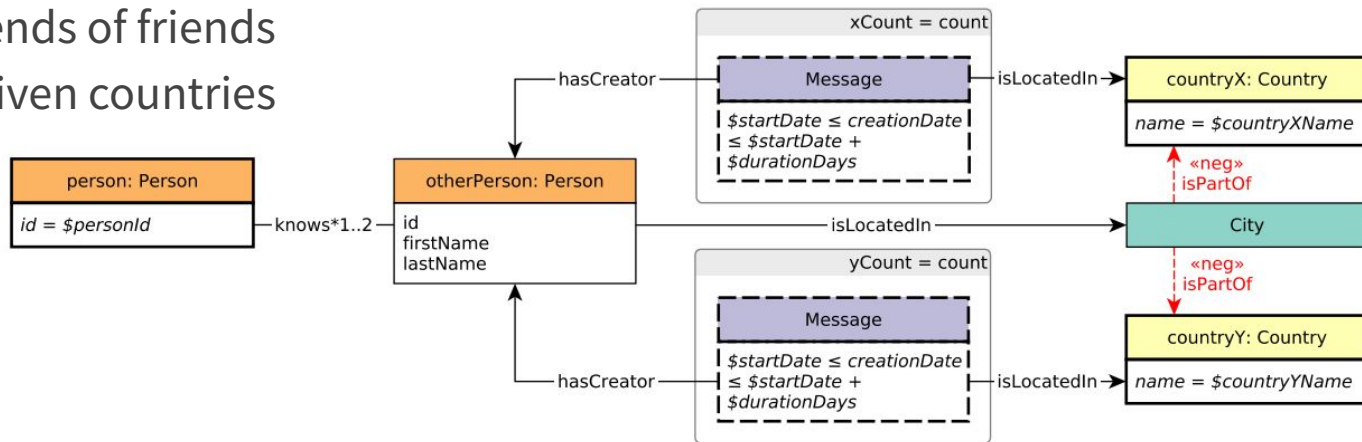
Short queries: Discover the neighbourhood of a Person or a Message node.



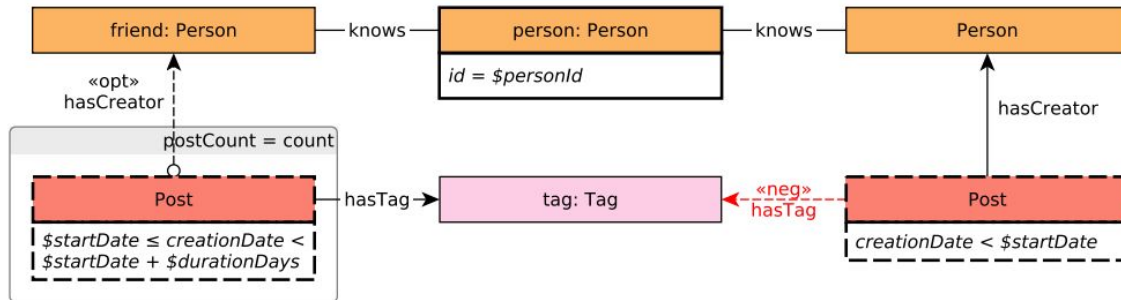
Update queries: Each query inserts a node or an edge.

Interactive workload: Complex queries

Q3: Friends and friends of friends that have been to given countries



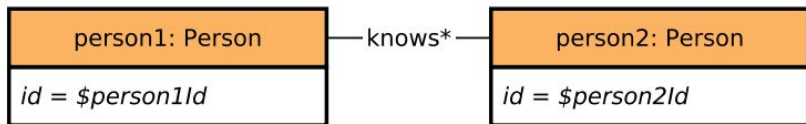
Q4: New topics



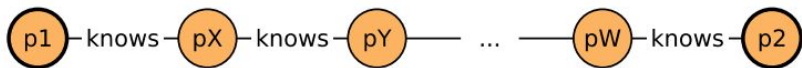
Interactive workload: Complex queries

Q14: Trusted connection paths

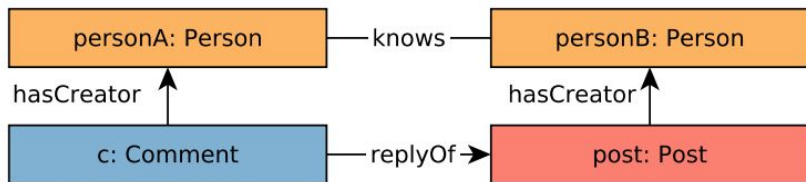
Enumerate all unweighted shortest paths on knows edges from person1 to person2.



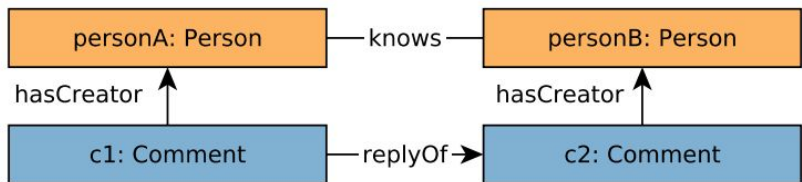
For each edge on the path, calculate a weight based on interactions between the pair of Persons of the edge, are calculated as a sum of cases #1 and #2 for the Persons (both ways), and the sum of these weights determine the total weight of each path.



Case 1: Replies on Posts, weight += 1.0 × count(c)



Case 2: Replies on Comments, weight += 0.5 × count(c1)



Interactive workload: Execution of queries

- **Update queries' issue times are taken from the update streams generated by the data generator.** These are the times where the actual event happened during the simulation of the social network.
- **Complex reads' times are expressed in terms of update operations.** For each complex read query type, a frequency value is assigned which specifies the relation between the number of updates performed per complex read.
- **For each complex read instance, a sequence of short reads is planned.** [...] The substitution parameters for short reads are taken from the results of previously executed complex reads and short reads. Once a short read sequence is issued (and provided that sufficient substitution parameters exist), there is a probability that another short read sequence is issued. This probability decreases for each new sequence issued. Since the same random number generator seed is used across executions, the workload is deterministic.

(See the [specification](#) for more details.)

Interactive workload: Data set

The Datagen produces 3 years worth of data. Of this data

- 90% is used the initial data set (separated into static/dynamic directories), and
- 10% is added later in the form of inserts (updates).

These inserts affect the entities in the “dynamic” category (e.g. Person/Message nodes, knows/likes edges). There are 8 insert operations, encoded in a variable-width CSV format:

- insert node: Person, Forum, Comment, Post
- insert edge: knows, hasMember, Comment-hasCreator-Person, Post-hasCreator-Person

$t \mid t_d \mid 1$	personId personFirstName personLastName gender birthday creationDate locationIP browserUsed cityId languages emails tagIds studyAt workAt
$t \mid t_d \mid 2$	personId postId creationDate
$t \mid t_d \mid 3$	personId commentId creationDate
$t \mid t_d \mid 4$	forumId forumTitle creationDate moderatorPersonId tagIds
$t \mid t_d \mid 5$	personId forumId creationDate
$t \mid t_d \mid 6$	postId imageFile creationDate locationIP browserUsed language content length authorPersonId forumId countryId tagIds
$t \mid t_d \mid 7$	commentId creationDate locationIP browserUsed content length authorPersonId countryId replyToPostId replyToCommentId tagIds
$t \mid t_d \mid 8$	person1Id person2Id creationDate

Interactive workload: driver #1

The driver has 3 main modes of operation:

1. Generate validation data set

- single-threaded, sequential execution
- inputs:
 - database with the initial data set loaded
 - query parameters: `substitution_parameters` dir 
 - update streams: `updateStream_*.csv` files
- output:
 - `validation_params.csv` file

2. Validate implementation

- single-threaded, sequential execution
- inputs:
 - database with the initial data set loaded
 - `validation_params.csv` file
- output:
 - pass/fail for each query execution



The substitution parameters contain datetime values as UNIX epochs in milliseconds (in GMT).



The default workload includes **updates**, therefore **reload the database between each run**.

Interactive workload: driver #2

3. Execute benchmark

- multi-threaded
- limited non-deterministic behaviour is possible due to concurrent execution
- inputs:
 - database with the initial data set loaded
 - query parameters: `substitution_parameters` directory
 - update streams: `updateStream_*.csv` files
 - TCR value
- output:
 - pass/fail
 - performance results

Parameter selection

For each generated data set, the Datagen component creates *substitution parameters* (also known as “query parameters” and “query seeds”). Parameters are selected so that the *variance* of the expected execution times is limited. This is a non-trivial task as graph queries are prone to high-variance due to their skewed, power-law degree distribution (exhibited by e.g. the Person-knows-Person subgraph).

Path queries are especially tricky as the execution time has huge variance based on whether the path exist (usually quick to find) or does not exist (usually slow to prove).

There is a txt file for each query and each line them corresponds to a query execution.



The datetime values in the txt files are represented as UNIX epoch values. The driver converts them into GMT-based timestamps.



[Parameter Curation for Benchmark Queries](#), TPCTC 2014

TCR and valid benchmark runs #1

Implementations compete on the TCR (total compression ratio), i.e. how quickly they can replay a sequence of operations. A TCR of 0.1 means the operations are played at 10x speed. **A lower TCR is better.**

For a run to count as a valid run, the implementation has sustain sufficient performance for 2 hours (after a 30-minute initial warmup whose performance results are discarded).



TCR and valid benchmark runs #2

For a given SF/TCR, implementations have to satisfy the 95% on-time requirement:

In order to have a valid run, 95% of the queries must meet the following condition:

$$actual_start_time - scheduled_start_time < 1 \text{ second}$$

That is, queries have to start in less than 1 second of their originally intended start time. If the system falls behind too much and less than 95% of the queries start on time, the run is invalid.

In these cases, the test executor is advised to reduce the TCR and start another run.

 Due to potentially noisy execution environments and the slight differences in individual runs (due to multi-threaded execution), it is recommended to leave a bit 'in reserve' when calibrating the TCR value.

BI workload

- Analytical queries
- Cyclic subgraphs
- Shortest paths
- Inserts/deletes

Business Intelligence workload

Scenario: Ad-hoc data analytics

Queries: 20 complex reads; refresh operations: 8 inserts, 8 deletes

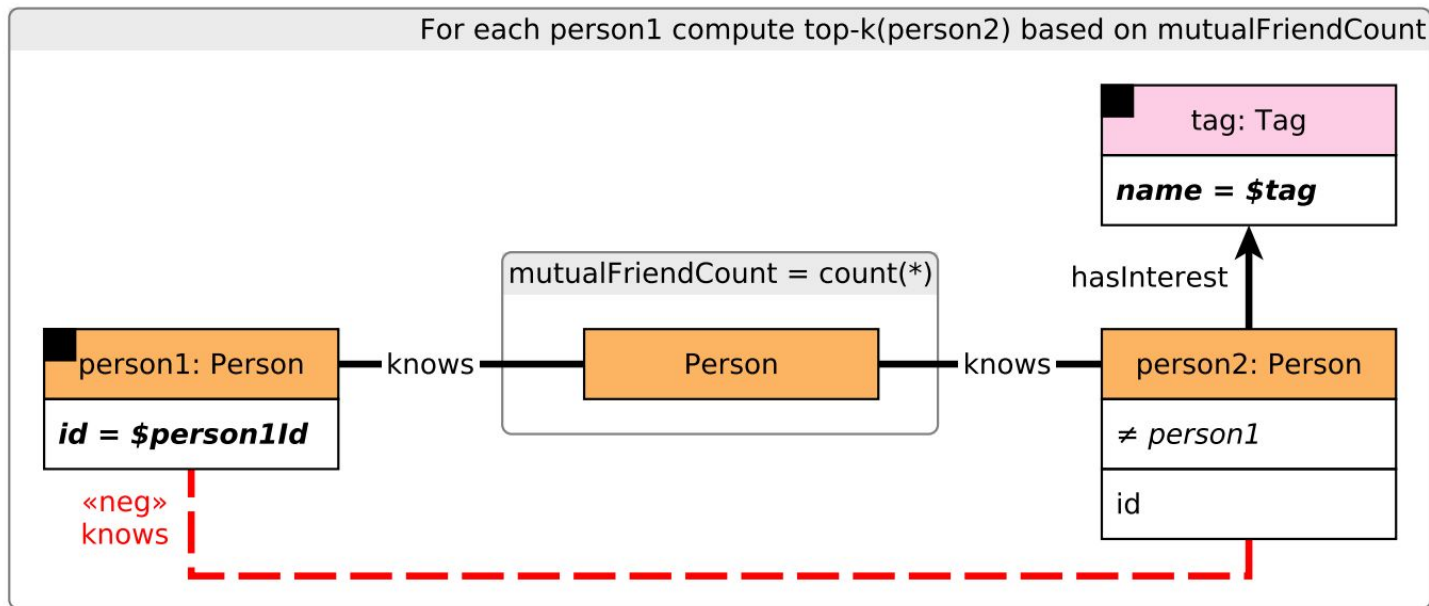
Audit rules:

- Must use a declarative query language unlike Interactive
- Defining views is allowed (the tradeoff is maintenance cost) same as Interactive

 Recently completed: Deprecated 10 queries and added 5 new ones

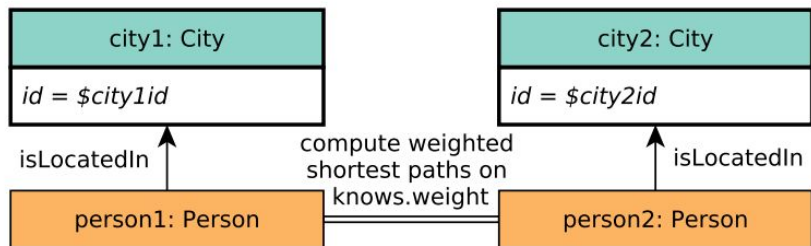
 SNB BI [[GRADES-NDA'18 paper](#)] [[GRADES-NDA'18 slides](#)] [[specification](#)]

Q18: Dense subgraphs for WCOJ algorithms

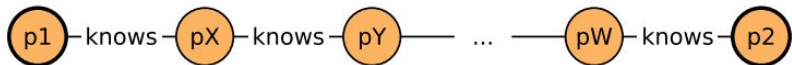


Q19: Weighted shortest paths

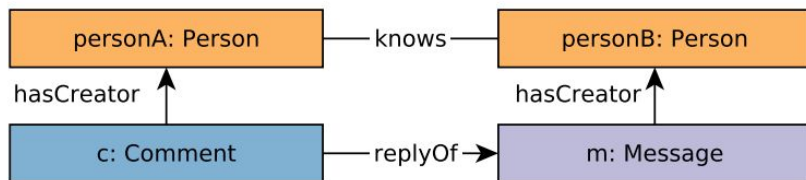
Find the shortest paths between all pairs of Persons in city1 and city2



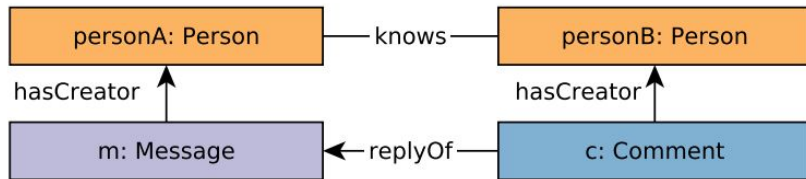
The weight of a knows edge is based on the number of interactions between its Persons:
 $\text{knows.weight} = 1 / (\text{count}(i1) + \text{count}(i2))$



Case i1: Reply from personA to Person B's Message



Case i2: Reply from personB to personA's Message



Parameter generator

Implemented in SQL, uses DuckDB :

- https://github.com/ldbc/ldbc_snb_data_converter/blob/main/paramgen.sh
- https://github.com/ldbc/ldbc_snb_data_converter/blob/main/factor-tables.sql
- https://github.com/ldbc/ldbc_snb_data_converter/blob/main/select-bi-params.sql

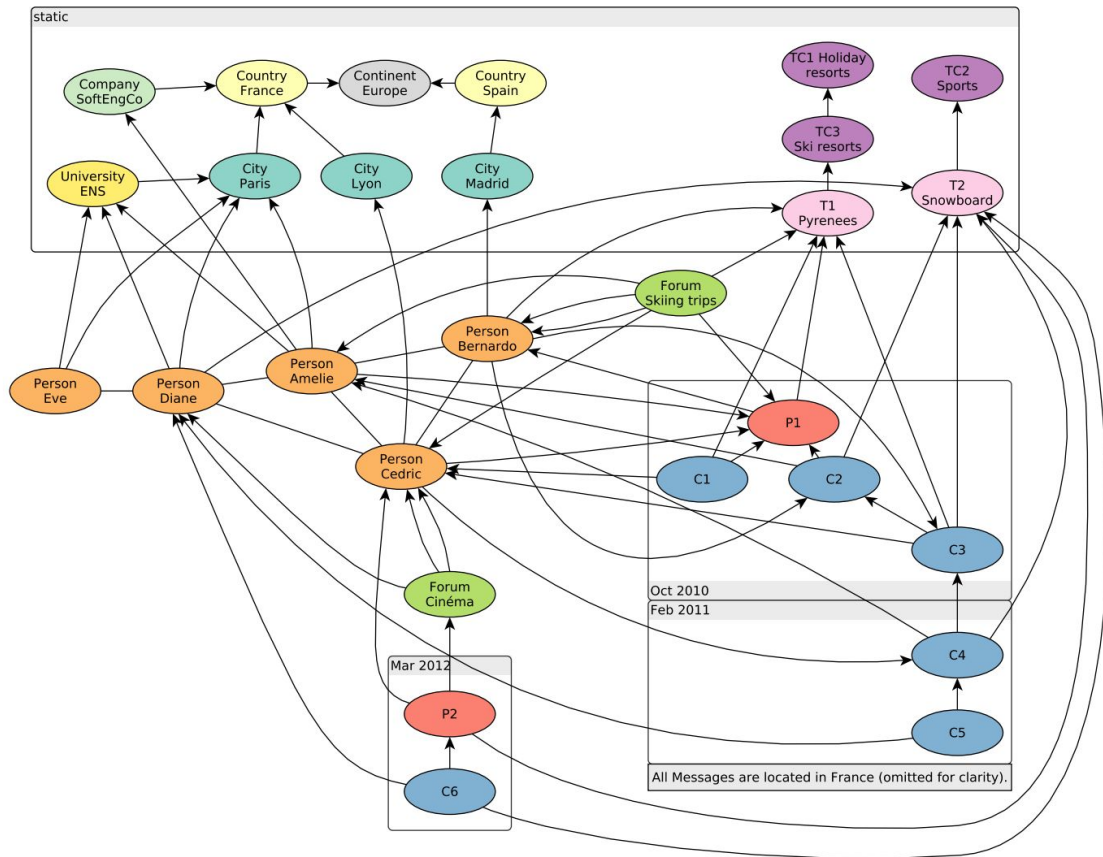
Example graph



Example graph

~30 nodes and ~60 edges

- Useful for **debugging the toolchain end-to-end.**
- Has at least one match for all BI queries.
- Parametrization for the Interactive workload to be done later.



Query design

Choke points and parameters

- Intended query plan
- Choke point analysis
- Parameter curation

Query templates and parameters

Queries are given using a **query template** which can have multiple **input parameters**.

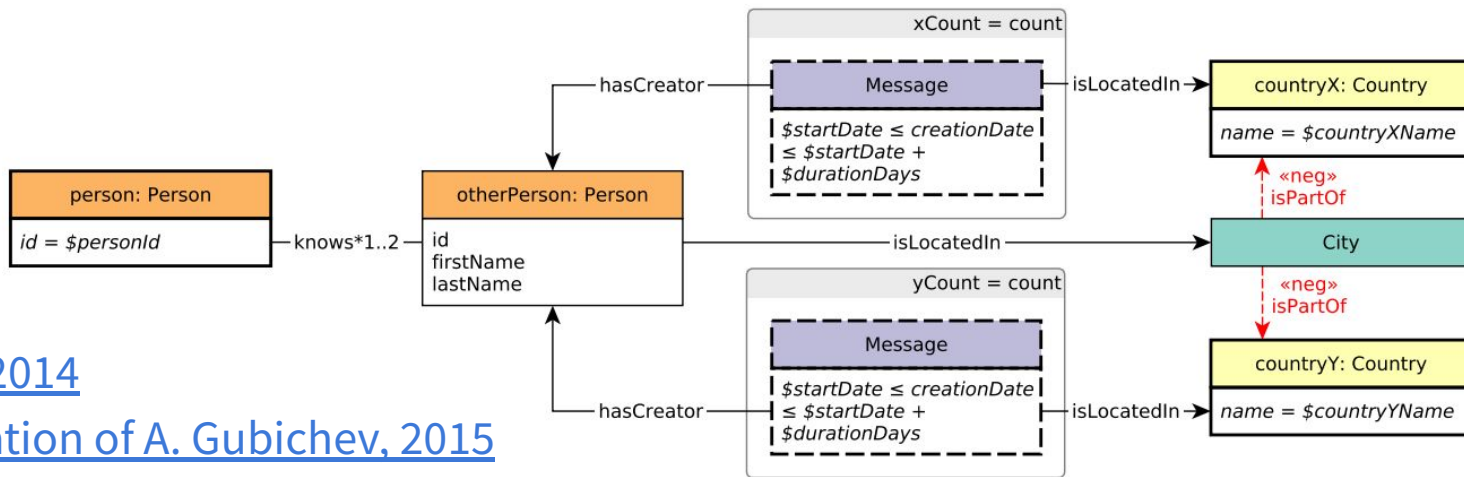
These are **substituted for with different parameters** during execution.

The parameters are produced by the **Paramgen** component of Datagen.

Intended query plan

The **intended query plan** of a query is the ideal execution plan to evaluate that query.
E.g. in Interactive Q3 params can be chosen to produce a large or a small result:

- Neighbouring countries X = Belgium Y = France
- Far away countries X = Mexico Y = Japan



Choke points

A choke point is a **difficult aspect of query processing** that has a significant impact on the performance of the query *when evaluated using the intended query plan*.

The TPCTC'12 paper analyzed TPC-H based on the lessons learnt when implementing the benchmark on Vectorwise, Virtuoso, and HyPer.

Examples:

- Join ordering
- Efficient antijoins and outer joins
- Handling paths

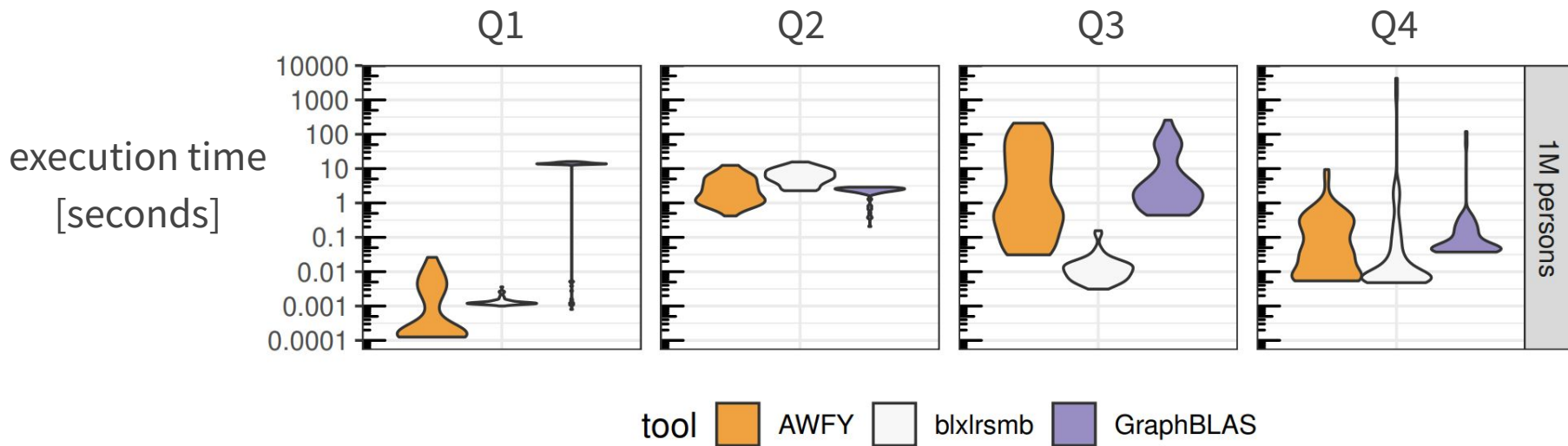


TPC-H choke points [[TPCTC'12](#)], Quantitative analysis of TPC-H CPs [[VLDB'20](#)]

Parameter curation

Goal: Reduce variance of query execution times, make results easier to interpret.

A negative example: Q1-Q4 of the SIGMOD 2014 Contest without parameter curation



Selecting entities for deletes

One can think of this as a special case of parameter curation: based on whether we select a Person

- with many friends and a lot of content or
- with little activity

The cost of performing the delete operation varies significantly.



This is currently being worked out as part of tuning the distribution of the deletes.

Ecosystem

- Specification
- Datagen
- Driver
- Implementations

Specification

Challenges:

- Coming up with a representative workload which has the “optimum” difficulty
- Define auditing rules (inspired by TPC)
- Define queries in an unambiguous way
- Define graphical notation (inspired by the graph transformation community)

We believe to have successfully tackled these in the latest specification.



[\[Latest on GitHub\]](#) [\[Stable on arXiv\]](#)

Driver

Lots of challenges regarding concurrent execution: tracking dependencies between refresh operations while maintaining a high throughput.

The driver implements these features in Java and is by far the largest project in SNB.

- main project: 38k LOC
- tests: 22k LOC

The new BI queries and deletes are already supported by the driver.

 Adding support for batched refresh operations is ongoing work.

 EU Deliverable “Benchmarking transactions” [[D2.2.3](#)]

Implementations

Complete implementations:

- PostgreSQL [SQL]
- Neo4j [Cypher]

Both of these are systems with an OLTP focus, thus geared towards Interactive.

Partial implementations:

- DuckDB [SQL]: implementation is in progress
- GraphBLAS [grblas/Python]
- ~~Stardog [SPARQL], Virtuoso [SQL], Hyper [SQL]~~ (dropped support)



A low-level (C/C++) imperative implementation would be useful for experimenting.

Auditing process

- Ensure objective comparison
- Drive competition among vendors

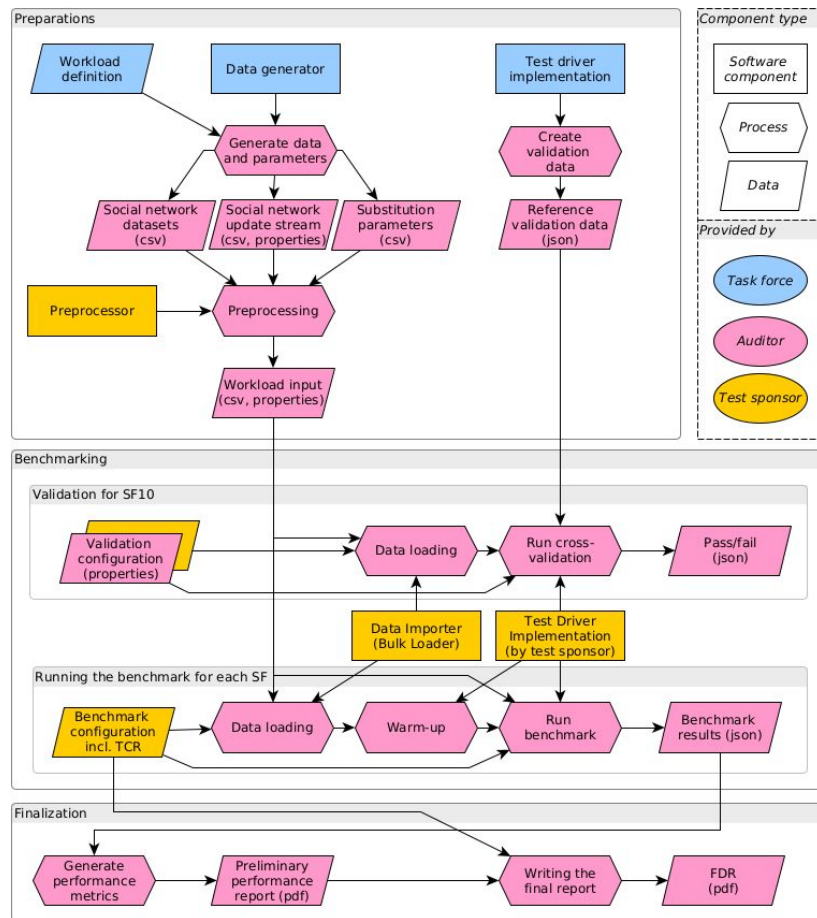
Auditing guidelines

Complex workflow to ensure fair comparison.

TPC has lots of rules to prevent cheating (including the use of “benchmark specials”), sometimes going as far as deprecating entire benchmarks.

Currently, there are such rules for **LD BC** but an audit process exist. Audited results:

- Are produced by an independent auditor
- Can be published online
- Spurious results can be challenged



ACID tests

Ensure that the isolation level claimed by the DBMS is enforced.  [TPCTC'20](#)

[illegible]

LDBC Lite

– proposal –

- Subgraph matching
- Dense subgraphs
- Outer joins
- Minimal attributes

LDBC Lite Microbenchmark Suite

Details in another presentation.

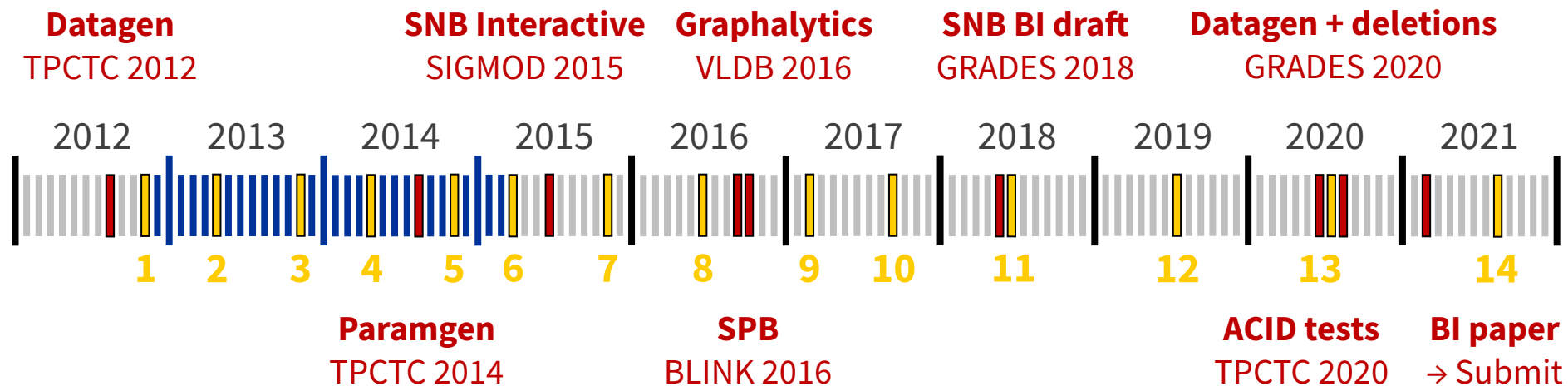
Why are the SNB Interactive/BI workloads so complex?

- Real graph data is correlated [TPCTC'12]
 - Graph data generator with correlations
 - Scalability is important -> distributed generator
 - Need to support multiple layouts (merged FK/projected FK)
- A mature database system has dozens of intertwined optimizations [TPCTC'13]
 - Characterized by choke points in the context of TPC-H
 - ~30 choke points (aggregation, join, data access locality, expressions, correlation, parallelism)
- Benchmark need parameterized queries [TPCTC'14]
 - Some warmup is required but many systems cache results -> queries needs to be parameterized
 - Parameter selection needs to be done carefully to make query times predictable
- Issuing updates needs a sophisticated driver [SIGMOD'15]
 - Update streams need to be able to run concurrently without cross-stream dependencies
- Updates are required to discourage read-only data structures [GRADES-NDA'20]
 - Without updates, materialization of partial results could give an unfair advantage
 - Introducing deletions needs lifespan management
- ACID compliance is required [TPCTC'20]
 - Needs a separate test suite as it is difficult to test within the full benchmark

Summary

- Timeline of benchmarks so far
- Future plans

LDBC project, benchmark papers & meetings



EU FP7 project | TUC meetings | Benchmark papers

Future work

 **Continuous:** We support the adoption of this benchmark and help audits

 **New benchmarks:** There are many possibilities to discover, including

- a simpler subgraph query benchmark (*update: done, this is the [LSQB](#)*)
- a benchmark with “financial fraud detection”-like queries
- streaming/temporal graph queries
- machine learning (embeddings, GNNs)
- mix of queries and analytics (see the recent analysis of [SIGMOD’14 Contest](#))
- queries on a more complex schema

We are happy to discuss proposed new graph benchmarks. Feel free to reach out at info@ldbcouncil.org



*The graph & RDF
benchmark reference*